

Programowanie obiektowe

Wykład 1

Wprowadzenie do programowania obiektowego

Problem: złożoność systemów

Każdy większy system informatyczny ma dwa problemy:

1. Liczbę elementów
2. Zależności między nimi

- 1000 linii kodu nie jest jeszcze problemem.
- 1000 wzajemnych zależności – już tak.

Największym wyzwaniem nie jest algorytm, lecz organizacja struktury systemu.

Program w podejściu strukturalnym

Program = algorytmy + struktury danych

- Algorytm opisuje co zrobić
- Struktura danych opisuje na czym operujemy

Typowy model:

- funkcje operujące na danych przekazywanych jako parametry

Ograniczenia tego modelu

W miarę wzrostu systemu:

- funkcji przybywa,
- danych przybywa,
- zależności rosną,
- odpowiedzialność się rozmywa.

Problem: kto odpowiada za poprawność danych?

Program w podejściu obiektowym

W OOP dane i operacje są połączone.

Zamiast:

- algorytmy + struktury danych

Mamy:

- obiekty realizujące odpowiedzialność.

Czym jest obiekt?

Obiekt to:

- jednostka programu posiadająca **tożsamość**,
- przechowująca **stan**,
- udostępniająca **operacje**.

Obiekt:

- kontroluje własne dane,
- reaguje na komunikaty (wywołania metod),
- realizuje określoną odpowiedzialność.

Obiekt to nie tylko dane – jest aktywnym elementem systemu.

Program to zbiór obiektów komunikujących się poprzez wywołania metod.

Czym jest metoda?

Metoda to operacja:

- należąca do klasy,
- wykonywana w kontekście konkretnego obiektu,
- mająca dostęp do jego stanu.

Różnica:

- funkcja operuje na przekazanych danych,

```
wypożycz(ksiazka, czytelnik);
```

- metoda operuje na stanie obiektu.

```
czytelnik.wypożycz(ksiazka);
```

W **OOP** operacja należy do obiektu.

Czym jest klasa?

Klasa to:

- definicja typu obiektowego,
- opis struktury (pola),
- opis zachowania (metody),
- specyfikacja odpowiedzialności.

Klasa określa:

- jakie dane posiada obiekt,
- jakie operacje udostępnia,
- jakie reguły obowiązują.

Klasa definiuje rolę w systemie.

Obiekt jest instancją **klasy**.

Paradygmat programowania

Sposób organizowania programu i odpowiedzialności w systemie.

Różne paradygmaty proponują różne sposoby organizacji:

- Imperatywny – wokół instrukcji
- Strukturalny – funkcji i danych
- Obiektowy – obiektów
- Funkcyjny – funkcji jako wartości

W tym kursie skupimy się na paradygmacie obiektowym.

Cztery filary programowania obiektowego

Programowanie obiektowe opiera się na:

- Abstrakcji – dotyczy projektowania klasy i interfejsu
- Hermetyzacji – dotyczy kontroli dostępu i odpowiedzialności
- Dziedziczeniu – dotyczy ponownego wykorzystania i hierarchii typów
- Polimorfizmie – dotyczy elastyczności i rozszerzalności systemu

Na pierwszych wykładach skupimy się głównie na abstrakcji i hermetyzacji:

- **Abstrakcja** – modelowanie istotnych cech obiektu i pomijanie szczegółów implementacyjnych.
- **Hermetyzacja** – Ukrywanie stanu obiektu i kontrolowanie dostępu do niego poprzez metody.

Języki używane na kursie

Uczymy się modelu obiektowego, nie konkretnego języka.

W trakcie zajęć będziemy używać:

- C++
- Java
- C#

Model obiektowy jest wspólny.

Różnice dotyczą głównie:

- zarządzania pamięcią,
 - składni,
 - środowiska uruchomieniowego.
- C++ - pełna kontrola nad pamięcią
 - Java - środowisko zarządzane (JVM, GC)
 - C# - środowisko .NET, zarządzana pamięć

Obiekt w praktyce

Przykład (Java):

```
String tekst = "Java";  
int dlugosc = tekst.length();
```

- `tekst` jest obiektem.
- `length()` jest metodą.
- Wywołujemy usługę oferowaną przez obiekt.

To samo podejście występuje w C++ i C#.

Odpowiedzialność w systemie – przykład

System biblioteczny

Zidentyfikujmy odpowiedzialności:

Książka

- przechowuje informacje o tytule, autorze, dostępności
- wie, czy jest wypożyczona

Czytelnik

- przechowuje dane osobowe
- może wypożyczyć książkę
- posiada listę aktualnych wypożyczeń

Wypożyczenie

- łączy książkę i czytelnika
- przechowuje datę wypożyczenia i termin zwrotu

Każdy obiekt ma:

- własne dane
- własne operacje
- własną odpowiedzialność

UML - Unified Modeling Language

Służy do opisu struktury systemu.

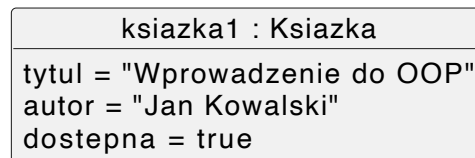
Diagram klas



- górna część - nazwa klasy
- środkowa - stan (pola)
- dolna - operacje (metody)

Więcej o UML na kolejnych wykładach

Diagram obiektu (instancji)



Klasa opisuje strukturę.

Obiekt jest konkretną instancją tej struktury.

Gdzie istnieje obiekt?

Obiekt w programie to nie tylko pojęcie logiczne

Obiekt:

- musi zostać utworzony,
- zajmuje miejsce w pamięci,
- ma określony czas życia,
- może zostać zniszczony.

Ten sam model obiektowy może mieć różne reprezentacje w pamięci w zależności od języka.

- C++ - programista zarządza pamięcią ręcznie,
- Java / C# - pamięcią zarządza środowisko uruchomieniowe (garbage collector).

Różnice implementacyjne nie zmieniają modelu obiektowego.

Poziomy abstrakcji w programie

Projektując system, możemy patrzeć na niego na różnych poziomach.

1. Poziom koncepcyjny (model)

- Jakie obiekty istnieją?
- Jakie mają odpowiedzialności?
- Jak są ze sobą powiązane?

To opisujemy np. w UML.

2. Poziom implementacyjny

- Jak reprezentujemy obiekt w kodzie?
- Jak wygląda klasa?
- Jak wywoływane są metody?

To zależy od języka.

3. Poziom pamięci i wykonania

- Gdzie obiekt istnieje?
- Jak długo żyje?
- Kto zarządza jego usunięciem?

To zależy od modelu pamięci języka.

Pojęcie zmiennej w programowaniu

- Zmienna to nazwana przestrzeń w pamięci, która przechowuje określoną wartość.
- Każda zmienna ma **typ** określający, jakie dane może przechowywać (np. `int` , `double` , `char`).
- **Deklaracja** – określenie nazwy i typu zmiennej.
 - *Przykład:* `int liczba;`
- **Inicjalizacja** – przypisanie wartości do zmiennej.
 - *Przykład:* `int liczba = 10;`
- **Zakres (scope)** – określa, gdzie zmienna jest dostępna (lokalna, globalna, instancyjna, statyczna).
- **Przechowywanie w pamięci** – zmienne mogą być przechowywane na **stosie** lub **stercie**.

Zakres zmiennej w programowaniu

- Zakres zmiennej (scope) określa, gdzie w programie dana zmienna jest widoczna i może być używana.
- Zmienne lokalne:
 - Deklarowane wewnątrz funkcji lub bloku `{}`.
 - Dostępne tylko w obrębie tej funkcji/bloku.
 - Po zakończeniu działania funkcji zmienna jest usuwana z pamięci.

```
#include <stdio.h>

int main() {
    int x = 10; // Zmienna globalna dla funkcji main()

    if(x > 0) {
        int y = 20; // Zmienna dostępna tylko w tym bloku
        printf("Wewnątrz bloku: %d\n", y);
    }

    printf("%d\n", y); // ❌ Błąd! Zmienna y nie istnieje poza blokiem
    return 0;
}
```

Przechowywanie zmiennych na stosie vs. sterście

■ Stos (*Stack*)

- Struktura LIFO (*Last In, First Out*) – ostatnia wartość dodana jest usuwana jako pierwsza.
- Używany do przechowywania **zmiennych automatycznych** (lokalne zmienne funkcji).
- Szybki dostęp, ale ograniczona ilość pamięci (może prowadzić do "stack overflow").
- Pamięć zwalniana automatycznie, gdy funkcja się kończy.

```
void funkcja() {  
    int x = 10; // Przechowywane na stosie  
} // "x" znika po zakończeniu funkcji
```

Przechowywanie zmiennych na stosie vs. stercie

■ Sterta (*Heap*)

- Pamięć dynamiczna, zarządzana ręcznie (`malloc()` / `free()` w C, `new` / `delete` w C++).
- Używana do przechowywania **zmiennych dynamicznych** (np. dużych obiektów, tablic).
- Większa pojemność niż stos, ale wolniejszy dostęp.
- Programista musi ręcznie zwolnić pamięć, inaczej powstaje "memory leak".

```
int* ptr = malloc(sizeof(int)); // Zmienna dynamiczna na stercie
free(ptr);                     // Ręczne zwolnienie pamięci
```

■ W języku C++

```
int* ptr = new int; // Zmienna dynamiczna na stercie
delete ptr;         // Ręczne zwolnienie pamięci
```

Zmienne wskaźnikowe w C/C++

- Zmienna wskaźnikowa (*pointer*) to zmienna, która przechowuje adres pamięci innej zmiennej.
- Wskaźniki pozwalają na dynamiczne zarządzanie pamięcią.
- Deklaracja wskaźnika

```
int* ptr; // Wskaźnik do zmiennej typu int`
```

- Przypisanie adresu (operator &)

```
int liczba = 42;  
ptr = &liczba; // Przypisanie adresu zmiennej 'liczba' do wskaźnika 'ptr'
```

- Dereferencja (operator `*`) – odczyt/zapis wartości pod adresem wskaźnika

```
std::cout << *ptr; // Odczyt wartości przechowywanej pod adresem wskaźnika  
*ptr = 100;        // Zmiana wartości zmiennej, na którą wskazuje wskaźnik
```

Referencja w C++

- Referencja (*reference*) to alternatywny sposób dostępu do zmiennej poprzez jej alias (drugą nazwę).
- Działa jak wskaźnik, ale nie wymaga operatorów `*` i `&` do dereferencji.
- Musi być zainicjalizowana przy deklaracji i nie może zmieniać obiektu, na który wskazuje.

```
int liczba = 10;  
int& ref = liczba; // Referencja do zmiennej 'liczba'
```

- Przekazywanie przez referencję pozwala na modyfikację zmiennej wewnątrz funkcji.

```
void zwieksz(int& liczba) {  
    liczba++; // Modyfikujemy oryginalną zmienną  
}  
  
int main() {  
    int x = 5;  
    zwieksz(x);  
    printf("Nowa wartość x: %d\n", x); // x == 6  
    return 0;  
}
```

Obiekty w C++

- W C++ programista ma wybór: obiekt dynamiczny / obiekt automatyczny
- **Obiekt automatyczny**
 - tworzy się poprzez deklarację
 - usuwany automatycznie po wyjściu z zakresu

```
{  
    Plik p;  
    p.pisz(1);  
}
```

Obiekty w C++

■ Obiekt dynamiczny

- tworzy się dynamicznie przez `new`
- reprezentowany przez **wskaźnik** (zmienna automatyczna)
- należy go jawnie usunąć przez `delete`

```
{  
    Plik *pP;  
    pP = new Plik;  
    pP->pisz(1);  
    delete pP;  
}
```


Obiekty w Javie

- W Javie programista nie ma wyboru:
 - **Obiekt dynamiczny**
 - tworzy się dynamicznie przez `new`
 - reprezentowany przez **zmienną referencyjną** (zmienna automatyczna)
 - nie usuwa się go (zajmuje się tym *garbage collector*)

```
{  
    Plik p;  
    p = new Plik();  
    p.pisz(1);  
}
```

Obiekty w C#

- W C# większość obiektów to **obiekty dynamiczne**
 - tworzy się dynamicznie przez `new`
 - reprezentowany przez **zmienną referencyjną** (zmienna automatyczna)
 - nie usuwa się go (zajmuje się tym *garbage collector*)

```
{  
    Plik p;  
    p = new Plik();  
    p.Pisz(1);  
}
```

Struktura programu głównego w C++

```
1  #include <iostream> // Biblioteka standardowa do wejścia i wyjścia
2
3  using namespace std; // Używanie przestrzeni nazw std (opcjonalne)
4
5  int main() {
6      cout << "Witaj w C++!" << endl; // Wyświetlenie tekstu na ekranie
7      return 0; // Zakończenie programu
8  }
```

- `#include <iostream>` - dyrektywa preprocesora dołączająca bibliotekę do obsługi wejścia/wyjścia.
- `using namespace std;` - pozwala uniknąć pisania `std::cout` , `std::endl` itp.
- `int main()` - funkcja główna musi zwracać `int` .
- `cout << "Witaj w C++!" << endl;` - instrukcja wyjściowa (odpowiednik `printf` z C).
- `return 0;` - zwraca wartość `0` , oznaczającą poprawne zakończenie programu.

Parametry funkcji `main()` w C++

- Podobnie jak w C, `main()` może przyjmować parametry:

```
1  #include <iostream>
2  using namespace std;
3
4  int main(int argc, char* argv[]) {
5      cout << "Liczba argumentów: " << argc << endl;
6      for (int i = 0; i < argc; i++) {
7          cout << "Argument " << i << ": " << argv[i] << endl;
8      }
9      return 0;
10 }
```

- Umożliwiają one przekazywanie argumentów z wiersza poleceń.
 - `argc` - liczba argumentów (pierwszy argument to nazwa programu)
 - `argv` - tablica ciągów znaków (`char*`)

Deklaracje funkcji w C++

- W C++ można deklarować funkcje przed `main()`, a definiować je poniżej.

```
1  #include <iostream>
2  using namespace std;
3
4  // Deklaracja funkcji
5  int add(int a, int b);
6
7  int main() {
8      cout << add(3, 5) << endl;
9      return 0;
10 }
11
12 // Definicja funkcji
13 int add(int a, int b) {
14     return a + b;
15 }
```

- Deklaracja informuje kompilator, że taka funkcja istnieje.
- Deklaracja jest konieczna, jeśli definicja funkcji znajduje się za `main()`.

Konwersja typów w C++

- C++ ma bardziej restrykcyjny system typów niż C.
- Ten kod zadziała w C bez ostrzeżenia:

```
1  #include <stdio.h>
2
3  int main() {
4      double x = 5.7;
5      int y = x;
6      printf("%d\n", y); // Wypisze 5 (utrata precyzji)
7      return 0;
8  }
```

- W C++ powinniśmy użyć operatora rzutowania `(int)` albo (lepiej) `static_cast<int>`:

```
1  #include <iostream>
2
3  int main() {
4      double x = 5.7;
5      int y = (int)x;
6      int z = static_cast<int>x;
7      std::cout << y << z << std::endl;
8      return 0;
9  }
```

Struktura programu głównego w Javie

- Każdy program w Javie zaczyna się od klasy, a wykonanie programu rozpoczyna się od metody `main()`, która działa jako punkt wejścia.

```
1  public class Main {  
2      public static void main(String[] args) {  
3          System.out.println("Witaj w Javie!");  
4      }  
5  }
```

- `public class Main` - deklaracja klasy
 - Każdy program w Javie musi znajdować się wewnątrz klasy.
 - Nazwa klasy musi odpowiadać nazwie pliku (Main.java).
- Metoda `main()`:
 - `public` - metoda jest publiczna, czyli dostępna z zewnątrz.
 - `static` - pozwala na uruchomienie bez tworzenia obiektu klasy.
 - `String[] args` - argumenty przekazywane z wiersza poleceń.
- `System.out.println()` - służy do wypisywania tekstu na ekran.

Deklaracje funkcji w Javie

- W Javie metody muszą należeć do klasy.
- W `main()` możemy wywoływać inne metody statyczne.

```
1  public class Main {  
2      // Metoda statyczna  
3      public static int add(int a, int b) {  
4          return a + b;  
5      }  
6  
7      public static void main(String[] args) {  
8          int c = add(3, 5); // Wywołanie metody  
9          System.out.println(c);  
10     }  
11 }
```

- Kolejność deklaracji `main()` i `add()` nie ma znaczenia.

Struktura programu głównego w C#

- W języku C# każdy program musi znajdować się wewnątrz klasy lub struktury, a jego wykonanie zaczyna się od metody `Main()`.

```
1  using System; // Import biblioteki standardowej
2
3  class Program {
4      static void Main() {
5          Console.WriteLine("Witaj w C#!");
6      }
7  }
```

- `using System;` - importujemy przestrzeń nazw `System`, aby móc używać `Console.WriteLine()`.
- `class Program` - deklaracja klasy
 - Każdy program w C# znajduje się w klasie.
 - Nazwa klasy może być dowolna.
- Metoda `Main()`:
 - `static` - metoda jest statyczna, więc można ją wywołać bez tworzenia obiektu.
- Instrukcja `Console.WriteLine()` - służy do wypisywania tekstu na ekran.

Deklaracje funkcji w C#

- W C#, podobnie jak w Javie, każda metoda musi należeć do klasy.
- Bez tworzenia obiektu możemy wywoływać metody statyczne.

```
1  using System;
2
3  class Program {
4      // Statyczna metoda
5      static int Add(int a, int b) {
6          return a + b;
7      }
8
9      static void Main() {
10         int c = Add(5, 7); // Wywołanie metody
11         Console.WriteLine(c);
12     }
13 }
```

- Kolejność definicji `Add()` i `Main()` jest dowolna.

Top-level statements w C#

- Od C# 9.0 wprowadzono *Top-level statements* (instrukcje na najwyższym poziomie), dzięki którym można pominąć deklarację klasy i metodę `Main()`.
- Od C# 10 domyślnie dodawane są tzw. *global usings*. Pozwala to pominąć również niektóre `using`.
- Minimalny program w C# 10+:

```
1 Console.WriteLine("Witaj w C#!");
```

- Kompilator sam tworzy ukrytą klasę i `Main()`.
- Działa to również dla funkcji:

```
1 static int Add(int a, int b) {  
2     return a + b;  
3 }  
4  
5 int c = Add(5, 7);  
6 Console.WriteLine(c);
```

- Dzięki temu kod wygląda jak w Pythonie czy JavaScript.
- Dobre rozwiązanie dla prostych skryptów lub nauki C#.

Wypisywanie na ekran w C++

- W **C++** do wypisywania tekstu na ekranie używamy `std::cout` z biblioteki `<iostream>`.

```
1  #include <iostream> // Dołączamy bibliotekę do obsługi wejścia/wyjścia
2
3  int main() {
4      std::cout << "Witaj w C++!" << std::endl;
5      return 0;
6  }
```

- `std::cout` - To standardowy strumień wyjściowy (ang. *console output*), który służy do wypisywania tekstu na ekran.
- Operator `<<` - Jest używany do przekazywania danych do `cout`, np.:
- `std::endl` - Przenosi kursor do nowej linii (tak jak `'\n'`), a dodatkowo opróżnia bufor wyjściowy.
- Można wypisywać wiele rzeczy naraz:

```
1  int liczba = 42;
2  std::cout << "Liczba: " << liczba << ", podwojona: " << liczba * 2 << std::endl;
```

Wypisywanie na ekran w Javie

- W **Java** do wypisywania tekstu używamy `System.out.println()` lub `System.out.print()`.

```
public class Main {  
    public static void main(String[] args) {  
        System.out.print("To jest ");           // Wypisuje bez nowej linii  
        System.out.println("Java!");           // Wypisuje i przechodzi do nowej linii  
    }  
}
```

- Aby wypisać kilka wartości, możemy połączyć je `+` - automatycznie konwertuje to liczby na tekst.

```
int wiek = 25;  
String imie = "Jan";  
System.out.println("Imię: " + imie + ", Wiek: " + wiek);
```

- Albo użyć `printf` do sformatowania wyjścia:

```
System.out.printf("Imię: %s, Wiek: %d\n", imie, wiek);
```

Wypisywanie na ekran w C#

- W **C#** standardowym sposobem wypisywania tekstu jest `Console.WriteLine()` lub `Console.Write()`.

```
Console.Write("To jest ");           // Wypisuje bez nowej linii
Console.WriteLine("C#!\n");         // Wypisuje i przechodzi do nowej linii
```

- Możemy łączyć wartości przy pomocy `+` (automatycznie konwertuje liczby na tekst).

```
int wiek = 25;
string imie = "Jan";
Console.WriteLine("Imię: " + imie + ", Wiek: " + wiek);
```

- Formatowanie podobnie jak w `printf`:

```
Console.WriteLine("Imię: {0}, Wiek: {1}", imie, wiek);
```

- Czytelniejszym sposobem jest interpolacja stringów (C# 6+):

```
Console.WriteLine($"Imię: {imie}, Wiek: {wiek}");
```

- `$"..."` pozwala na wstawianie zmiennych bez `+`.

Wczytywanie danych z klawiatury w C++

- W C++ do wczytywania danych z klawiatury używamy `std::cin` z biblioteki `<iostream>`.

```
int liczba;  
  
std::cout << "Podaj liczbę: ";  
std::cin >> liczba;
```

- `cin` to standardowy strumień wejściowy (*console input*).
- `>>` (operator ekstrakcji) – Pobiera dane z wejścia (`cin`) i zapisuje do zmiennej.
- Wczytywanie wielu wartości jednocześnie:

```
int a, b;  
  
std::cout << "Podaj dwie liczby oddzielone spacją: ";  
std::cin >> a >> b;
```

Wczytywanie tekstu z klawiatury w C++

- Wczytywanie tekstu:

```
std::string imie;  
  
std::cout << "Podaj swoje imię: ";  
std::cin >> imie;
```

- `cin` domyślnie wczytuje tylko jedno słowo (ignoruje spacje).
- Aby wczytać całe zdanie, używamy `std::getline()`.

```
std::string imie;  
  
std::cout << "Podaj swoje pełne imię i nazwisko: ";  
std::getline(std::cin, imie);
```

- `std::getline()` wczytuje cały wiersz.

Wczytywanie danych z klawiatury w Javie

- Do pobierania danych od użytkownika najczęściej używa się klasy `Scanner` (z pakietu `java.util`).

```
import java.util.Scanner; // Importujemy klasę Scanner
```

- `Scanner` inicjujemy standardowym wejściem: `System.in`.

```
Scanner scanner = new Scanner(System.in); // Tworzymy obiekt Scanner
```

- Możemy wczytywać pojedyncze tokeny:

```
String imie = scanner.next(); // wczytuje jedno słowo  
int wiek = scanner.nextInt(); // wczytuje jedną liczbę
```

- Albo całą wiersz:

```
String imie = scanner.nextLine(); // Wczytuje cały wiersz
```

- Zamknięcie skanera (dobry nawyk):

```
scanner.close();
```

Wczytywanie danych z klawiatury w C#

- W C# do pobierania danych od użytkownika używamy `Console.ReadLine()`, które zwraca wartość jako `string`.

```
// Pobranie tekstu
Console.Write("Podaj swoje imię: ");
string imie = Console.ReadLine(); // Wczytuje cały wiersz
```

- Jeśli chcemy wczytać liczbę, musimy przekonwertować tekst na odpowiedni typ:

```
// Pobranie liczby (konwersja string -> int)
Console.Write("Podaj swój wiek: ");
int wiek = int.Parse(Console.ReadLine()); // Zamiana string na int

Console.WriteLine($"Witaj, {imie}! Masz {wiek} lat.");
```

- `int.Parse()` - Konwertuje wczytany string na liczbę całkowitą.

Tablice w C++

- Posługiwanie się tablicami w C++ nie różni się od C.

```
int liczby[5] = {1, 2, 3, 4, 5};  
  
for (int i = 0; i < 5; i++) {  
    std::cout << liczby[i] << " ";  
}
```

- Pętla *range-based for* (C++11+) oferuje przyjaźniejszą składnię iterowania przez tablicę (ale jedynie statyczną – nie działa dla wskaźników):

```
for (int x : liczby) {  
    cout << x << " ";  
}
```

Tablice dynamiczne typów prostych w C++

- Tablice dynamiczne w C++ pozwalają na tworzenie tablic o zmiennym rozmiarze w czasie działania programu.
- Do ich obsługi używamy operatora `new` do alokacji pamięci i operatora `delete[]` do jej zwalniania.

```
int rozmiar;  
cout << "Podaj rozmiar tablicy: ";  
cin >> rozmiar;  
  
int* tablica = new int[rozmiar]; // Dynamiczna alokacja tablicy  
  
// posługujemy się tablicą...  
  
delete[] tablica; // Zwolnienie pamięci
```

Tablica obiektów i wskaźników na obiekty w C++

- W C++ można tworzyć tablice obiektów oraz tablice wskaźników na obiekty.
- Sposób zarządzania nimi zależy od tego, czy używamy tablicy statycznej czy dynamicznej.
- **Tablica obiektów (statyczna)** – Jeśli znamy liczbę obiektów z góry, możemy użyć zwykłej tablicy:

```
Samochod auta[3];
```

- **Tablica dynamiczna obiektów** – Jeśli nie znamy liczby obiektów wcześniej, możemy użyć `new[]` :

```
Samochod* auta = new Samochod[n];
```

- Tak stworzoną tablicę musimy ręcznie usunąć (zwolnić pamięć) przy pomocy `delete[]` .

```
delete[] auta;
```

Tablica obiektów i wskaźników na obiekty w C++

- **Tablica wskaźników na obiekty** – Zamiast przechowywać obiekty, możemy przechowywać wskaźniki na obiekty.
- Dynamicznie tworzymy tablicę wskaźników:

```
Samochod** auta = new Samochod*[n];
```

- Wypełniamy ją dynamicznie stworzonymi obiektami:

```
auta[i] = new Samochod(marka, rok);
```

- Zwalniając pamięć, musimy usunąć zarówno obiekty (np. w pętli):

```
for (int i = 0; i < n; i++) {  
    delete auta[i];  
}
```

- Jak i tablicę:

```
delete[] auta;
```

Tablica typów prostych w Javie

- Deklaracja i inicjalizacja

```
int[] liczby = new int[5]; // Tworzy tablicę o 5 elementach (domyślnie 0)
```

- Alternatywnie

```
int[] liczby = {1, 2, 3, 4, 5};
```

- Posługiwanie się tablicą:

```
// Iteracja po tablicy
for (int i = 0; i < liczby.length; i++) {
    System.out.println("Indeks " + i + ": " + liczby[i]);
}
```

- Tablica sama pamięta swój rozmiar (`length`).
- Domyślne wartości: 0 dla `int` , `false` dla `boolean` , `null` dla obiektów.

Tablica obiektów w Javie

- Tablica typów referencyjnych (np. `Point[]`)
- Domyślnie zawiera wskaźniki (referencje) na `null`

```
Point[] punkty = new Point[3];
```

- Alternatywnie - tworzenie tablicy zainicjowanej:

```
Point[] punkty = { new Point(1, 2), new Point(3, 4), new Point(5, 6) };
```

- Korzystanie z tablicy:

```
// Iteracja i wywoływanie metod wartości
for (int i = 0; i < punkty.length; i++) {
    punkty[i].translate(1, -1);
}
```


Iteracja w Javie

- Pętla typu *for-each* służy do iterowania po elementach tablicy w prostszy sposób niż klasyczna pętla `for`.
- W Javie pętla *for-each* działa dla tablic i kolekcji (`ArrayList`, `HashSet` itp.).

```
int[] liczby = {10, 20, 30, 40, 50};  
  
// Pętla for-each  
for (int liczba : liczby) {  
    System.out.println(liczba);  
}
```

Tablica typów prostych w C#

- W C# tablice działają bardzo podobnie do Javy.

```
// Deklaracja i inicjalizacja
int[] liczby = new int[5]; // Tworzy tablicę o 5 elementach (domyślnie 0)

// Przypisanie wartości
liczby[0] = 10;
liczby[1] = 20;
liczby[2] = 30;

// Alternatywna inicjalizacja
int[] liczby2 = { 1, 2, 3, 4, 5 };

// Iteracja po tablicy
for (int i = 0; i < liczby.Length; i++) {
    Console.WriteLine($"Indeks {i}: {liczby[i]}");
}
```

- W C# używamy `Length` zamiast `length` w Javie.

Tablica obiektów w C#

- Tablica typów referencyjnych (`Point[]`)

```
Point[] punkty = new Point[3]; // Tworzenie tablicy obiektów (domyślnie null)
punkty[0] = new Point(1, 2);   // Inicjalizacja obiektów
punkty[1] = new Point(3, 4);
punkty[2] = new Point(5, 6);
```

- Albo:

```
Point[] punkty = { new Point(1, 2), new Point(3, 4), new Point(5, 6) };
```

- Iteracja i wywołanie metody

```
for (int i = 0; i < punkty.Length; i++) {
    punkty[i].Translate(-1, 1);
}
```

Iteracja w C#

- W C# `foreach` działa dla tablic i kolekcji (`List<>` , `HashSet<>`).

```
int[] liczby = {10, 20, 30, 40, 50};
```

```
// Pętla foreach
```

```
foreach (int liczba in liczby) {  
    Console.WriteLine(liczba);  
}
```

Napisy jako obiekty - klasa `string` w C++

- Klasa `std::string` (z nagłówka `<string>`) to bardziej funkcjonalny i bezpieczniejszy niż tradycyjne `char` sposób pracy z ciągami znaków.

- Podstawy użycia

```
string imie = "Jan";
```

- Konkatenacja (`+` i `+=`)

```
imie += "ek";
```

- Odczyt długości `size()`

```
cout << imie.size() << endl;
```

- Dostęp do znaków (`[]`)

```
cout << imie[0] << endl;    // 'J'
```

- Porównywanie stringów: `==` , `!=` , `<` , `>` , znajdowanie podciągów: `find()` , wycinanie napisu: `substr()` , modyfikowanie napisu: `insert()` , `replace()`

Napisy jako obiekty - klasa `String` w Javie

- W Javie `String` jest klasą (nie typem prostym), dlatego operacje na `String` wykorzystują metody obiektowe.
- Tworzenie i używanie `String`

```
String tekst = "Java";  
System.out.println(tekst.length()); // 4  
System.out.println(tekst.charAt(1)); // 'a'  
System.out.println(tekst.toUpperCase()); // "JAVA"
```

- Inne metody: `contains()` sprawdza czy string zawiera frazę, `indexOf()` zwraca pozycję podciągu, `substring()` wycina fragment, `replace()` zamienia fragment, `split()` dzieli na tablicę:

```
String tekst = "Java,Python,C#";  
String[] jezyki = tekst.split(",");
```

- **Uwaga:**
 - W Javie porównywanie `String` trzeba robić odpowiednio: `text.equals(inny_tekst)`
 - W Javie `String` jest **niemutowalny** – oznacza to, że nie można zmieniać zawartości istniejącego stringa, a każda operacja modyfikująca tworzy nowy obiekt w pamięci.

Napisy jako obiekty - klasa `string` w C#

- W C# `string` jest niemutowalny, podobnie jak `String` w Javie.
- Tworzenie i używanie `string`

```
string tekst = "CSharp";  
Console.WriteLine(tekst.Length); // 6  
Console.WriteLine(tekst.ToUpper()); // "CSHARP"  
Console.WriteLine(tekst.Contains("Sh")); // true
```

- Inne: `IndexOf()` poszukuje pierwszego wystąpienia, `Substring()` wycina fragment tekstu, `Replace()` zmienia fragment tekstu, `Split()` dzieli na tablicę:

```
string tekst = "C#,Python,Java";  
string[] jezyki = tekst.Split(',');
```

- W C# można porównywać stringi przy pomocy operatorów: `==`, `<`, `>`.

Zapis do pliku tekstowego w C++

- Zapis do pliku tekstowego oferuje w C++ klasa `std::ofstream` (*output file stream*) z biblioteki `<fstream>`.

```
#include <iostream>
#include <fstream> // Do obsługi plików

int main() {
    std::ofstream plik("dane.txt"); // Otwieramy plik do zapisu

    plik << "Witaj w pliku!\n";
    plik << "To jest przykład zapisu do pliku w C++.\n";

    plik.close(); // Zamykamy plik
    std::cout << "Dane zapisane do pliku!" << std::endl;

    return 0;
}
```

- Tworzymy obiekt `std::ofstream`.
- Używamy `<<` (jak w `std::cout`) do zapisu.
- Zamykamy plik metodą `close()`.

Odczyt z pliku tekstowego w C++

- Odczyt z pliku tekstowego oferuje w C++ klasa `std::ifstream` (*input file stream*) z biblioteki `<fstream>`.
- Czytamy dane za pomocą `>>` (dla pojedynczych słów/liczb) lub `getline()` (dla całych linii).

```
#include <iostream>
#include <fstream>

int main() {
    std::ifstream plik("liczby.txt");
    if (!plik) {
        std::cout << "Błąd otwarcia pliku!" << std::endl;
        return 1;
    }

    int liczba;
    while (plik >> liczba) { // Czytamy po jednej liczbie
        std::cout << "Przeczytano liczbę: " << liczba << std::endl;
    }

    plik.close();
    return 0;
}
```

Zapis do pliku tekstowego w Javie

- `BufferedWriter` zapewnia wydajny i efektywny zapis do pliku tekstowego:

```
import java.io.*;

public class Main {
    public static void main(String[] args) throws IOException {
        BufferedWriter writer = new BufferedWriter(new FileWriter("plik.txt"));
        writer.write("Linia 1");
        writer.newLine(); // Dodaje nową linię
        writer.write("Linia 2");
        writer.close();
        System.out.println("Dane zapisane do pliku!");
    }
}
```

- Java wymaga specyfikowania wyjątków wejścia/wyjścia:
 - na razie najprostszy sposób by sobie z tym poradzić, to umieszczenie w nagłówku funkcji `main()` deklaracji `throws IOException`.

Zapis do pliku tekstowego w Javie

- `PrintWriter` jest łatwiejszy do użycia i obsługuje metody `print()` i `println()`.

```
import java.io.*;

public class Main {
    public static void main(String[] args) throws IOException {
        PrintWriter writer = new PrintWriter("plik.txt");
        writer.println("Nowy tekst");
        writer.println("Jeszcze jedna linia");
        writer.close();
        System.out.println("Dane zapisane!");
    }
}
```

Odczyt z pliku tekstowego w Javie

- `BufferedReader` to wydajny odczyt w większych blokach.

```
import java.io.*;

public class Main {
    public static void main(String[] args) throws IOException {
        BufferedReader reader = new BufferedReader(new FileReader("plik.txt"));
        String linia;
        while ((linia = reader.readLine()) != null) { // Czytanie linia po linii
            System.out.println(linia);
        }
        reader.close();
    }
}
```

Odczyt z pliku tekstowego w Javie

- `Scanner` jest prostszy w użyciu, szczególnie do czytania liczb lub słów.

```
import java.io.File;
import java.io.FileNotFoundException;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) throws FileNotFoundException {
        Scanner scanner = new Scanner(new File("plik.txt"));
        while (scanner.hasNextInt()) { // Czytanie liczba po liczbie
            int liczba = scanner.nextInt();
            System.out.println("Odczytana liczba: " + liczba);
        }
        scanner.close();
    }
}
```

Zapis do pliku tekstowego w C#

- C# również oferuje różne sposoby obsługi plików
 - `StreamReader` / `StreamWriter` - efektywne operacje na plikach.
 - Statyczne metody klasy `File` - najprostsza opcja dla małych plików.
- Zapis przy pomocy `StreamWriter`

```
StreamWriter writer = new StreamWriter("plik.txt"); // Otwieramy plik do zapisu
writer.WriteLine("Linia 1");
writer.WriteLine("Linia 2");
writer.Close();
Console.WriteLine("Dane zapisane do pliku!");
```

- Prostszą w użyciu alternatywę stanowią (statyczne) metody klasy `File` :

```
File.WriteAllText("plik.txt", "Pierwsza linia\nDruga linia");
```

- Zapis tablicy stringów (jako kolejnych linii):

```
string[] linie = { "Linia 1", "Linia 2", "Linia 3" };
File.WriteAllLines("plik.txt", linie);
```

Odczyt z pliku tekstowego w C#

- Odczyt z pliku z użyciem klasy `StreamReader`

```
StreamReader reader = new StreamReader("plik.txt");
string linia;
while ((linia = reader.ReadLine()) != null) { // Czytanie linia po linii
    Console.WriteLine(linia);
}
reader.Close();
```

- Metoda `File.ReadAllText()` pozwala odczytać całą zawartość pliku jako jeden string.

```
string zawartosc = File.ReadAllText("plik.txt");
```

- Metoda `File.ReadAllLines` odczytuje wszystkie linie jako tablicę stringów.

```
string[] linie = File.ReadAllLines("plik.txt");
Console.WriteLine("\nLinie pliku:");
foreach (string linia in linie) {
    Console.WriteLine(linia);
}
```